

11. Dynamic Programming (High Frequency)

Module 1: DP Theory – Core Concepts

1.1 What is Dynamic Programming?

Dynamic Programming (DP) is an optimization technique for solving problems with **overlapping subproblems** and **optimal substructure**.

Two key properties:

1. **Overlapping Subproblems:** The same smaller subproblems occur multiple times in the recursion tree.
2. **Optimal Substructure:** An optimal solution to the problem can be constructed from optimal solutions to its subproblems.

1.2 DP State Design & Transitions

State ($dp[i]$ or $dp[i][j]$): Represents the answer for a specific subproblem.

Transition: A recurrence relation that defines how to compute $dp[i]$ using previously computed states.

Base Case: The simplest subproblem(s) that can be solved directly.

1.3 Two Approaches: Memoization vs Tabulation

Aspect	Memoization (Top-Down)	Tabulation (Bottom-Up)
Approach	Recursive + caching	Iterative + table
Space	O(n) recursion stack + table	O(n) table (or less)
Implementation	Easier to write from brute-force	More efficient, avoids recursion overhead
Use case	When only some states are needed	When all states need to be computed

Example – Fibonacci (Memoization):

```
int fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n-1) + fib(n-2);
}
```

Example – Fibonacci (Tabulation):

```
int fib(int n) {
    int[] dp = new int[n+1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

Module 2: Identifying Overlapping Subproblems

2.1 How to Recognize DP Problems

Look for:

- **Counting** the number of ways (e.g., unique paths, coin change).
- **Optimization** (maximize/minimize) – e.g., min cost, max profit.
- **Decision** problems (e.g., can we achieve target sum?).

Common patterns:

- **1D DP:** Fibonacci, Climbing Stairs, House Robber.
 - **2D DP:** Grid paths, Edit Distance, LCS.
 - **Knapsack variants:** 0/1 Knapsack, Coin Change, Subset Sum.
 - **String DP:** LCS, Edit Distance, Palindrome Partitioning.
 - **Matrix DP:** Matrix Chain Multiplication, Egg Dropping.
-

Module 3: 0/1 Knapsack

3.1 Problem Statement

Given weights $w[i]$ and values $v[i]$ of n items, and a knapsack capacity $\bar{w}$, maximize the total value by selecting items such that total weight $\leq \bar{w}$. Each item can be taken **at most once** (0/1).

3.2 DP State & Transition

State: $dp[i][j]$ = maximum value using first i items with capacity j .

Transition:

- **Skip item i :** $dp[i][j] = dp[i-1][j]$
- **Take item i (if $w[i] \leq j$):** $dp[i][j] = \max(dp[i-1][j], v[i] + dp[i-1][j - w[i]])$

3.3 Code – Tabulation

```
public static int knapsack(int[] weights, int[] values, int capacity, int n)
```

```

int n = weights.length;
int[][] dp = new int[n + 1][W + 1];

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= W; j++) {
        if (weights[i-1] <= j) {
            dp[i][j] = Math.max(dp[i-1][j], values[i-1] + weights[i-1]);
        } else {
            dp[i][j] = dp[i-1][j];
        }
    }
}
return dp[n][W];
}

```

Complexity: $O(n \times W)$ time, $O(n \times W)$ space.

3.4 Space-Optimized (1D DP)

Since `dp[i]` only depends on `dp[i-1]`, we can use a single array and iterate **backwards** to avoid overwriting.

```

public static int knapsackOptimized(int[] weights, int[] values) {
    int[] dp = new int[W + 1];
    for (int i = 0; i < weights.length; i++) {
        for (int j = W; j >= weights[i]; j--) {
            dp[j] = Math.max(dp[j], values[i] + dp[j - weights[i]]);
        }
    }
    return dp[W];
}

```

Time: $O(n \times W)$, **Space:** $O(W)$.

3.5 Visual Trace

Input: `weights = [2, 3, 4]`, `values = [3, 4, 5]`, `W = 5`

i\j	0	1	2	3	4	5
0	0	0	0	0	0	0
1 (w=2,v=3)	0	0	3	3	3	3
2 (w=3,v=4)	0	0	3	4	4	7
3 (w=4,v=5)	0	0	3	4	5	7

Result: $dp[3][5] = 7$ (take items 1 and 2: weight $2+3=5$, value $3+4=7$).

Module 4: Coin Change

4.1 Problem Statement

Given coin denominations and a target amount `amount`, find the **minimum** number of coins needed to make that amount (unlimited supply of each coin). If impossible, return -1.

4.2 DP State & Transition

State: $dp[i]$ = minimum coins needed for amount `i`.

Transition: $dp[i] = \min(dp[i - \text{coin}] + 1)$ for each coin.

Base case: $dp[0] = 0$.

4.3 Code – Tabulation

```
public static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
```

```

        for (int coin : coins) {
            if (i >= coin) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1)
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}

```

Complexity: $O(\text{amount} \times \text{coins})$, Space: $O(\text{amount})$.

4.4 Number of Ways – Coin Change II

Problem: Count the number of ways to make the amount using unlimited coins (order doesn't matter).

State: $dp[i]$ = number of ways to make amount i .

Transition: For each coin, update $dp[i] += dp[i - \text{coin}]$ (to avoid permutations, iterate coins outer loop).

```

public static int change(int amount, int[] coins) {
    int[] dp = new int[amount + 1];
    dp[0] = 1;
    for (int coin : coins) {
        for (int i = coin; i <= amount; i++) {
            dp[i] += dp[i - coin];
        }
    }
    return dp[amount];
}

```

Module 5: Subset Sum

5.1 Problem Statement

Given an array `nums` and a target `sum`, determine if there exists a subset whose sum equals the target.

5.2 DP State & Transition

State: `dp[i][j]` = true if using first `i` elements, sum `j` is achievable.

Transition:

- Skip: `dp[i][j] = dp[i-1][j]`
- Take: `dp[i][j] = dp[i-1][j - nums[i-1]]` (if `j >= nums[i-1]`)

5.3 Code – Space-Optimized (1D)

```
public static boolean subsetSum(int[] nums, int target) {
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;
    for (int num : nums) {
        for (int j = target; j >= num; j--) {
            dp[j] = dp[j] || dp[j - num];
        }
    }
    return dp[target];
}
```

Visual Trace: `nums = [1, 3, 4]`, `target = 6`

j	0	1	2	3	4	5	6
start	T	F	F	F	F	F	F
after 1	T	T	F	F	F	F	F
after 3	T	T	F	T	T	F	F
after 4	T	T	F	T	T	T	F? wait: j=6: dp[6]

Actually, for sum 6: `dp[6]` (from 1+? no) let's recalc: after 3, `dp[3]=T`, `dp[4]=T`. After 4: `j=4: T` (already T), `j=5: dp[5]||dp[1] = F||T = T`, `j=6:`

$dp[6][2] = F \parallel F = F$. So 6 is not achievable. But $1+? 3+? \rightarrow 1+?$
Actually $1+? 1+? 1+? = 6$ with $1+? 3+? 4+? = 8$? $1+? 3+? 4+?$ let's test
 $1+? 1+? 1+? 3+? = 6$? $1+1+?$ no duplicates not allowed. So false.

Module 6: Longest Common Subsequence (LCS)

6.1 Problem Statement

Given two strings `s1` and `s2`, find the length of the longest subsequence common to both.

6.2 DP State & Transition

State: $dp[i][j]$ = LCS length of `s1[0..i-1]` and `s2[0..j-1]`.

Transition:

- If `s1[i-1] == s2[j-1]`: $dp[i][j] = 1 + dp[i-1][j-1]$
- Else: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

6.3 Code – Tabulation

```
public static int lcs(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1.charAt(i-1) == s2.charAt(j-1)) {
                dp[i][j] = 1 + dp[i-1][j-1];
            } else {
                dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
}
```



```

        }
    }
    return dp[m][n];
}

```

6.4 Space-Optimized (1D)

Since `dp[i][j]` depends only on the **current** and **previous** row, we can use two arrays or one array with careful updates.

```

public static int lcsOptimized(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    int[] prev = new int[n + 1];
    int[] curr = new int[n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1.charAt(i-1) == s2.charAt(j-1)) {
                curr[j] = 1 + prev[j-1];
            } else {
                curr[j] = Math.max(prev[j], curr[j-1]);
            }
        }
        int[] temp = prev;
        prev = curr;
        curr = temp;
    }
    return prev[n];
}

```

Complexity: $O(m \times n)$ time, $O(n)$ space.

6.5 Visual Trace

Input: `s1 = "ABC", s2 = "AC"`

i\j	0	1(A)	2(C)
0	0	0	0
1(A)	0	1	1
2(B)	0	1	1
3(C)	0	1	2

Result: 2 ("AC").

Module 7: Longest Increasing Subsequence (LIS)

7.1 Problem Statement

Given an array, find the length of the longest subsequence that is strictly increasing.

7.2 DP State & Transition

State: $dp[i]$ = length of LIS ending at index i .

Transition: $dp[i] = 1 + \max(dp[j])$ for all $j < i$ and $nums[j] < nums[i]$.

Base case: $dp[i] = 1$ for all i .

7.3 Code – $O(n^2)$ DP

```
public static int lis(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
```

```

int maxLen = 1;

for (int i = 1; i < n; i++) {
    for (int j = 0; j < i; j++) {
        if (nums[j] < nums[i]) {
            dp[i] = Math.max(dp[i], dp[j] + 1);
        }
    }
    maxLen = Math.max(maxLen, dp[i]);
}
return maxLen;
}

```

7.4 $O(n \log n)$ Greedy + Binary Search

Idea: Maintain a list `tails` where `tails[i]` is the smallest tail of an increasing subsequence of length `i+1`. Use binary search to replace elements.

```

public static int lisOptimized(int[] nums) {
    List<Integer> tails = new ArrayList<>();
    for (int num : nums) {
        int pos = Collections.binarySearch(tails, num);
        if (pos < 0) pos = -pos - 1;
        if (pos == tails.size()) tails.add(num);
        else tails.set(pos, num);
    }
    return tails.size();
}

```

Complexity: $O(n \log n)$ time, $O(n)$ space.

Module 8: Edit Distance (Levenshtein Distance)

8.1 Problem Statement

Given two strings `word1` and `word2`, find the minimum number of operations (insert, delete, replace) to convert `word1` to `word2`.

8.2 DP State & Transition

State: `dp[i][j]` = min operations to convert `word1[0..i-1]` to `word2[0..j-1]`.

Transition:

- If `word1[i-1] == word2[j-1]`: `dp[i][j] = dp[i-1][j-1]`
- Else: `dp[i][j] = 1 + min(dp[i-1][j] (delete), dp[i][j-1] (insert), dp[i-1][j-1] (replace))`

Base cases: `dp[i][0] = i` (delete `i` chars), `dp[0][j] = j` (insert `j` chars).

8.3 Code – Tabulation

```
public static int editDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i-1) == word2.charAt(j-1)) {
                dp[i][j] = dp[i-1][j-1];
            } else {
                dp[i][j] = 1 + Math.min(dp[i-1][j],
                                         Math.min(dp[i][j-1], dp[i-1][j-1]))
            }
        }
    }
}
```

```

        }
    }
    return dp[m][n];
}

```

Space-Optimized (1D) is also possible using two rows.

Module 9: Palindrome Partitioning

9.1 Problem Statement

Given a string s , partition it such that every substring is a palindrome. Find the minimum number of cuts needed for palindrome partitioning.

9.2 DP Approach

Idea: Use two DP arrays:

- $isPal[i][j] = \text{true}$ if $s[i..j]$ is palindrome.
- $dp[i] = \text{min cuts for prefix } s[0..i]$.

Transition for dp: $dp[i] = \min(dp[i], dp[j-1] + 1)$ for all $j \leq i$ where $isPal[j][i]$ is true.

9.3 Code

```

public static int minCut(String s) {
    int n = s.length();
    boolean[][] isPal = new boolean[n][n];
    int[] dp = new int[n];

    // Precompute palindromes
    for (int i = 0; i < n; i++) isPal[i][i] = true;
}

```

```

for (int len = 2; len <= n; len++) {
    for (int i = 0; i + len - 1 < n; i++) {
        int j = i + len - 1;
        if (s.charAt(i) == s.charAt(j) && (len == 2 || isPal[i][j])) {
            isPal[i][j] = true;
        }
    }
}

// DP for min cuts
for (int i = 0; i < n; i++) {
    if (isPal[0][i]) {
        dp[i] = 0;
    } else {
        dp[i] = i;
        for (int j = 1; j <= i; j++) {
            if (isPal[j][i]) {
                dp[i] = Math.min(dp[i], dp[j-1] + 1);
            }
        }
    }
}
return dp[n-1];
}

```

Module 10: 2-D Grid & Matrix DP

10.1 Unique Paths

Problem: Count paths from top-left to bottom-right moving only right/down.

State: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$

Base: $dp[0][j] = dp[i][0] = 1$

Code (space-optimized):

```
public int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] = dp[j] + dp[j-1];
        }
    }
    return dp[n-1];
}
```

10.2 Minimum Path Sum

Problem: Find min sum from top-left to bottom-right.

State: $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$

Code (space-optimized):

```
public int minPathSum(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[] dp = new int[n];
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (i == 0 && j == 0) dp[j] = grid[i][j];
            else if (i == 0) dp[j] = grid[i][j] + dp[j-1];
            else if (j == 0) dp[j] = grid[i][j] + dp[i];
            else dp[j] = grid[i][j] + Math.min(dp[j], dp[i]);
        }
    }
    return dp[n-1];
}
```

Module 11: Advanced DP Problems

11.1 Matrix Chain Multiplication

Problem: Given dimensions of matrices, find the minimum number of scalar multiplications to multiply the chain.

State: $dp[i][j]$ = min multiplications to multiply matrices $i..j$.

Transition: $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + \text{dims}[i-1] * \text{dims}[k] * \text{dims}[j])$ for k in $[i, j-1]$.

Code:

```
public static int matrixChain(int[] dims) {
    int n = dims.length - 1;
    int[][] dp = new int[n][n];
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            dp[i][j] = Integer.MAX_VALUE;
            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k+1][j] + dims[i] *
                    dims[k+1] * dims[j+1];
                dp[i][j] = Math.min(dp[i][j], cost);
            }
        }
    }
    return dp[0][n-1];
}
```

11.2 Egg Dropping

Problem: Given k eggs and n floors, find the minimum number of trials needed to determine the critical floor.

State: $dp[k][n]$ = min trials with k eggs and n floors.

Transition: $dp[k][n] = 1 + \min(\max(dp[k-1][x-1], dp[k][n-x]))$ for x in $[1, n]$.

[n-x])) for x in [1, n].

Optimized using Binary Search (or DP).

```
public static int eggDrop(int k, int n) {
    int[][] dp = new int[k+1][n+1];
    for (int i = 1; i <= k; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == 1) dp[i][j] = j;
            else if (j == 1) dp[i][j] = 1;
            else {
                dp[i][j] = Integer.MAX_VALUE;
                for (int x = 1; x <= j; x++) {
                    int res = 1 + Math.max(dp[i-1][x-1],
                        dp[i][j] = Math.min(dp[i][j], res);
                }
            }
        }
    }
    return dp[k][n];
}
```

Module 12: Hands-on – Knapsack Variants, LIS, LCS, Grid DP

12.1 Knapsack Variants

- **0/1 Knapsack** – covered above.
- **Unbounded Knapsack** – unlimited items (code similar to coin change II).
- **Minimum Cost to Fill Bag** – similar to unbounded, but min cost.

12.2 LIS Variants

- **Number of LIS** – count the number of increasing subsequences of max length.
- **Longest Bitonic Subsequence** – LIS from left and right.

12.3 LCS Variants

- **Longest Common Substring** – contiguous version, different transition.
- **Shortest Common Supersequence** – $\text{length} = m + n - \text{LCS}$.

12.4 Grid DP Variants

- **Obstacle Grid** – add obstacle check.
 - **Dungeon Game** – reverse DP from bottom-right.
-

Module 13: Optimization Techniques – Space & Time

13.1 Space Optimization

- **1D DP:** Replace 2D table with 1D/2 rows when transitions only depend on previous row/column.
- **Rolling Array:** Use `prev` and `curr` arrays.
- **In-place Modifications:** Sometimes the grid itself can be modified (e.g., min path sum).

13.2 Time Optimization

- **Binary Search:** Used in LIS $O(n \log n)$ and Egg Dropping.
 - **Greedy + DP:** Some problems combine greedy with DP.
 - **Divide and Conquer DP Optimization:** For problems with quadrangle inequality.
-

Module 14: Summary Table

Problem	State	Transition	Time	Space
Fibonacci	$dp[i]$	$dp[i-1] + dp[i-2]$	$O(n)$	$O(1)$
0/1 Knapsack	$dp[i][j]$	$\max(dp[i-1][j], v + dp[i-1][j-w])$	$O(nW)$	$O(W)$
Coin Change	$dp[i]$	$\min(dp[i-coin] + 1)$	$O(n \cdot \text{amount})$	$O(\text{amount})$
Subset Sum	$dp[i][j]$	$dp[i-1][j]$ OR $dp[i-1][j-\text{num}]$	$O(n \cdot \text{target})$	$O(\text{target})$
LCS	$dp[i][j]$	$1 + dp[i-1][j-1]$ if match else $\max$	$O(mn)$	$O(\min(m, n))$
LIS	$dp[i]$	$1 + \max(dp[j])$	$O(n^2)$ or $O(n \log n)$	$O(n)$
Edit Distance	$dp[i][j]$	$\min(\text{insert, delete, replace}) + 1$	$O(mn)$	$O(\min(m, n))$
Palindrome Partition	$dp[i]$	$\min(dp[j-1] + 1)$ if $\text{pal}[j][i]$	$O(n^2)$	$O(n^2)$
Unique Paths	$dp[i][j]$	$dp[i-1][j] + dp[i][j-1]$	$O(mn)$	$O(n)$
MCM	$dp[i][j]$	$\min(dp[i][k] + dp[k+1][j] + \text{dims})$	$O(n^3)$	$O(n^2)$

Module 15: Practice Exercises (15 problems)

1. **0/1 Knapsack** – Solve with space optimization.

2. **Coin Change II** – Number of ways (unlimited).
3. **Subset Sum** – Check if subset exists.
4. **Partition Equal Subset Sum** – Determine if array can be split into two equal sum subsets.
5. **LCS** – Print the actual subsequence.
6. **LIS** – Return the actual sequence.
7. **Edit Distance** – With space optimization.
8. **Minimum Insertion Steps to Make a String Palindrome** – $(n - \text{LPS length})$.
9. **Palindrome Partitioning II** – Min cuts (covered).
10. **Unique Paths II** – With obstacles.
11. **Minimum Path Sum** – With obstacles.
12. **Dungeon Game** – LeetCode 174.
13. **Matrix Chain Multiplication** – Also print parenthesization.
14. **Egg Dropping** – Binary search optimization.
15. **Longest Common Substring** – DP with max variable.